LAPORAN PRAKTIKUM

IOT 2

MODUL 7



Disusun Oleh:

Nama : Reza Aurelia Saputri

NPM : 25782080

Prodi : TRI C

Dosen:

Agus Ambarwari, S.Pd., M.Kom.

PROGRAM STUDI TEKNOLOGI REKAYASA INTERNET

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI LAMPUNG

2025/2026

7.5.2 Praktikum 2: Migrasi Simpul NodeMCU ESP8266 ke Broker MQTT Lokal

Skema Rangkaian Perangkat Keras:

Rangkaian perangkat keras pada praktikum ini identik dengan konfigurasi pada Modul 6 (hanya menggunakan sensor DHT dan indikator LED):

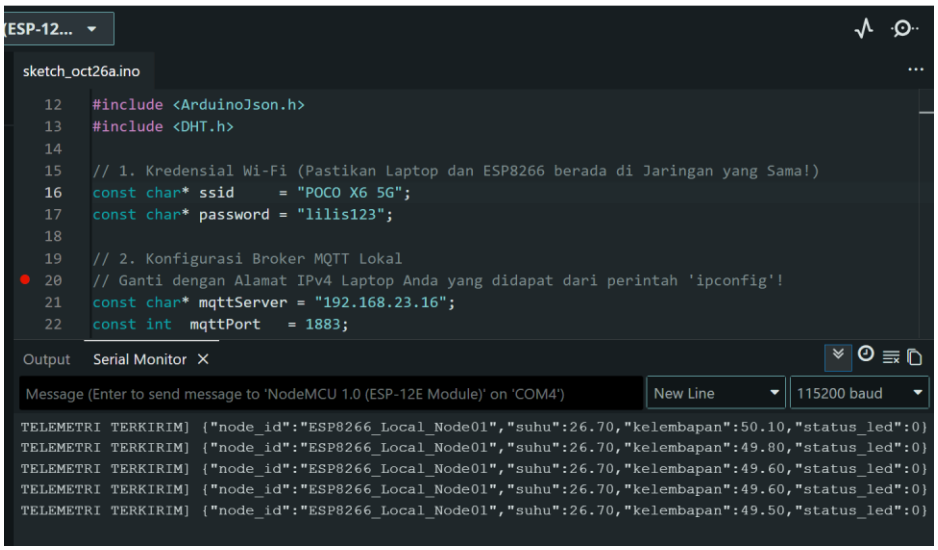
- Sensor DHT11/DHT22:
    - Pin VCC →Pin 3V3 NodeMCU
    - Pin GND →Pin GND NodeMCU
    - Pin Data →Pin D4 (GPIO 2)
- LED Indikator Aktuator:
    - Anoda (kaki panjang) →Pin D1 (GPIO 5)
    - Katoda (kaki pendek) →Resistor 220 Ω →Pin GND

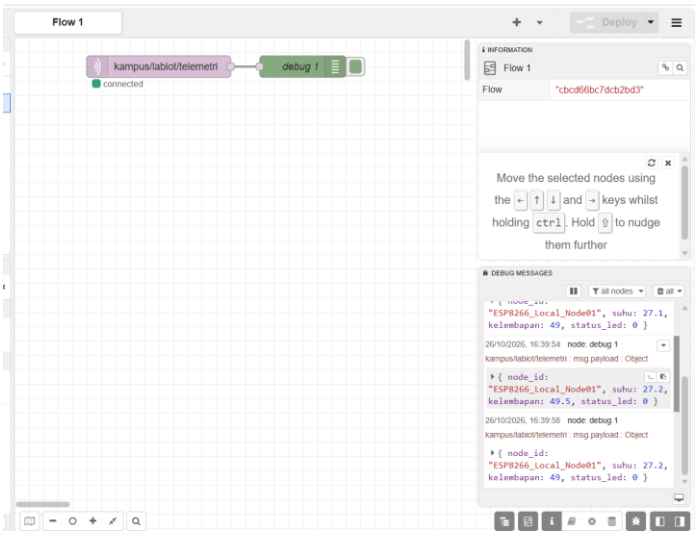
Langkah Kerja:

1. Buka peramban web dan akses antarmuka Node-RED di **http://localhost:1880**.
2. **Menambahkan Konfigurasi Server MQTT Broker:**
    - Di sebelah kiri (*Palette*), temukan kategori **network**, lalu klik dan seret (*drag*) node **mqtt in** ke tengah kanvas kerja.
    - Klik ganda (*double-click*) pada node mqtt in tersebut untuk membuka panel pengaturannya:
        - Pada baris **Server**, klik ikon pensil (📝) untuk membuat konfigurasi broker baru.
        - **Name:** Mosquitto Local
        - **Server:** Ketik **mqtt**
          (PENTING: Jangan gunakan localhost atau IP laptop! Karena Node-RED dan Mosquitto berada dalam satu jaringan Docker Compose yang sama, kontainer Node-RED dapat memanggil broker secara langsung menggunakan nama servis deklaratifnya, yaitu mqtt).
        - **Port:** 1883
        - Klik tombol merah **Add** di pojok kanan atas dialog.
3. **Mengonfigurasi Parameter Topik:**
    - Pada baris **Topic**, masukkan: kampus/labiot/telemetri
    - Pada baris **Output**, ubah dari *auto-detect* menjadi **a parsed JSON object**. (Fitur ini secara otomatis mengurai teks string JSON menjadi objek JavaScript hierarkis tanpa perlu menulis fungsi *parser* manual).
    - Klik tombol **Done**.
4. **Menambahkan Node Pemantau (*Debug Node*):**
    - Pada *Palette* sebelah kiri di kategori **common**, seret node **debug** ke kanvas.

- Tarik garis (*wire*) dari titik port keluaran di sisi kanan node mqtt in ke port masukan di sisi kiri node debug.

5. **Menerapkan Alur Kerja (*Deploy*):**
   - Klik tombol merah **Deploy** di sudut kanan atas antarmuka.
   - Amati bahwa di bawah node mqtt in muncul teks indikator hijau bertuliskan **connected**.
6. **Menganalisis Aliran Data:**
   - Klik tab ikon kumbang (🐞) di bilah sisi kanan (*Sidebar*) untuk membuka panel **Debug Messages**.
   - Anda akan melihat objek JSON masuk setiap 4 detik secara *real-time*:
   - kampus/labiot/telemetri : msg.payload : Object
   - {
   - node_id: "ESP8266_Local_Node01",
   - suhu: 29.8,
   - kelembapan: 62.4,
   - status_led: 0
   - }
   - Klik tanda panah segitiga kecil di samping teks Object untuk mengeksplorasi hierarki pasangan kunci dan nilai data.





7.5.4 Praktikum 4: Implementasi Rule Engine Otomasi Aktuator Loop Tertutup

Pada praktikum ini, kita akan membangun *Closed-Loop Rule Engine* otonom:

- **JIKA** nilai suhu yang dibaca oleh sensor melebihi ambang batas kritis 31.0°C: Node-RED secara otomatis memublikasikan paket JSON {"led": 1} untuk menyalakan LED indikator peringatan bahaya di pin D1.
- **JIKA** suhu berada dalam batas normal (≤ 31.0°C): Node-RED memublikasikan paket JSON {"led": 0} untuk memadamkan LED indikator.

Langkah Kerja:

1. **Membuat Node Percabangan Logika (switch):**
    - Pada kategori **function**, seret node **switch** ke kanvas.
    - Hubungkan port keluaran node mqtt in ke port masukan node switch.
    - Klik ganda node switch untuk menyetel aturan logika evaluasi:
        - **Property:** Ganti msg.payload menjadi **msg.payload.suhu**.
        - **Aturan 1 (Kondisi Panas):** Pilih operator **>**, lalu masukkan angka: **31**.
        - Klik tombol **+ add** di bagian bawah untuk menambahkan aturan kedua.
        - **Aturan 2 (Kondisi Aman):** Pilih operator **<=**, lalu masukkan angka: **31**.
        - Klik **Done**. Perhatikan bahwa node switch kini memiliki dua port keluaran bernomor 1 (atas) dan 2 (bawah).
2. **Membuat Node Transformasi Muatan Panas (change):**

- Seret node **change** dari kategori *function* ke kanvas di sebelah kanan atas node switch.
- Hubungkan port keluaran 1 (suhu > 31) dari node switch ke masukan node change pertama ini.
- Klik ganda node change:
  - **Name:** Nyalakan LED Indikator
  - **Set:** msg.payload
  - **To:** Ubah tipe data dari string (az) menjadi **JSON** ({ }), lalu masukkan nilai:
  - {
  - "led": 1

}

  - Klik **Done**.

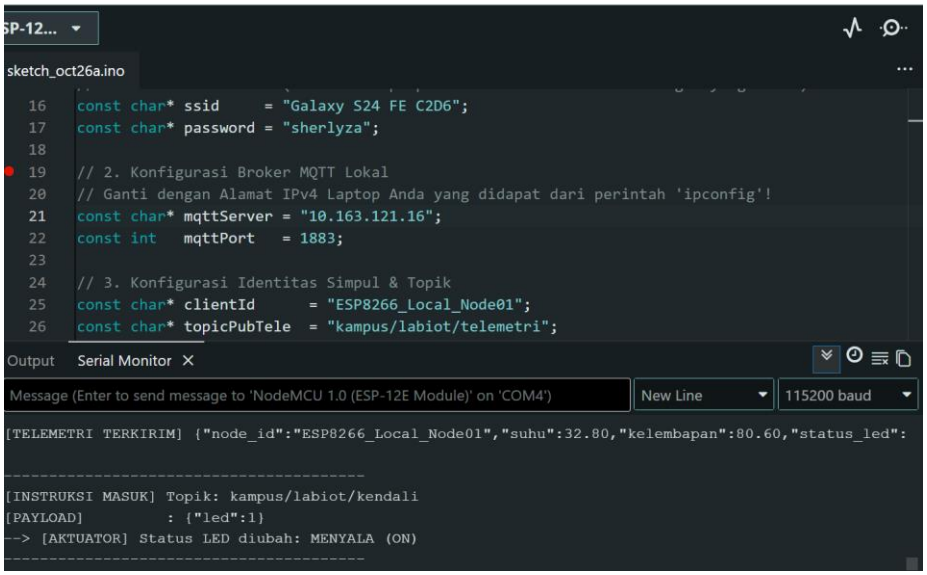
3. **Membuat Node Transformasi Muatan Normal (change):**
   - Seret node **change** kedua ke kanvas di sebelah kanan bawah node switch.
   - Hubungkan port keluaran 2 (suhu ≤ 31) dari node switch ke masukan node change kedua ini.
   - Klik ganda node change:
     - **Name:** Padamkan LED Indikator
     - **Set:** msg.payload
     - **To:** Ubah tipe data menjadi **JSON** ({ }), lalu masukkan nilai:
     - {
     - "led": 0

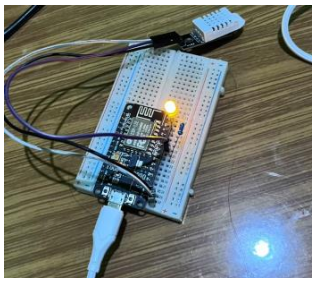
}

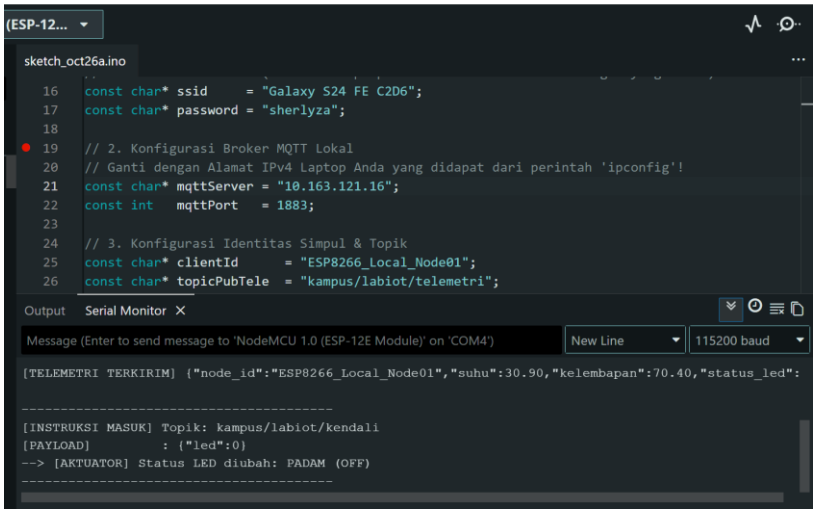
     - Klik **Done**.
4. **Menerbitkan Instruksi Balik ke Broker (mqtt out):**
   - Pada kategori **network**, seret node **mqtt out** ke kanvas.
   - Hubungkan keluaran dari node change pertama dan node change kedua secara bersamaan ke port masukan node mqtt out.
   - Klik ganda node mqtt out:
     - **Server:** Pilih Mosquitto Local.
     - **Topic:** kampus/labiot/kendali

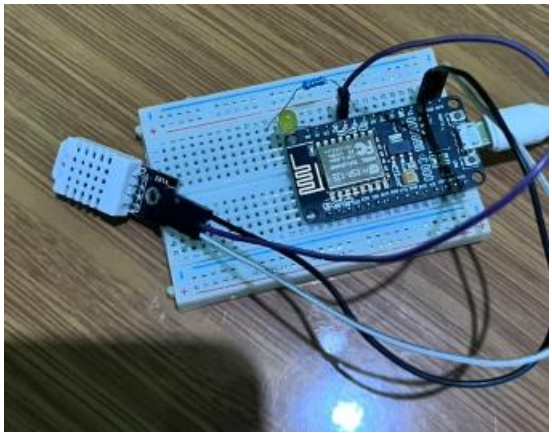
- **QoS:** 0 (atau 1)
- **Retain:** false
- Klik **Done**.

5. **Menerapkan Perubahan (*Deploy*):**
   - Klik tombol merah **Deploy** di kanan atas antarmuka.
6. **Pengujian Fisik Sistem Otomasi Tertutup:**
   - Amati status awal: Karena suhu ruangan umumnya berkisar antara $26°C - 29°C (\leq 31°C)$, LED indikator berada dalam kondisi padam.
   - Dekatkan sumber panas ringan (seperti ujung jari Anda yang ditempelkan kuat pada sensor DHT, atau hembusan napas hangat) hingga pembacaan suhu pada panel Debug merangkak naik melewati 31.0°C.
   - **Hasil Pengamatan:** Seketika suhu melampaui 31.0°C, Node-RED mengeksekusi port 1, mengirimkan payload {"led": 1}, dan LED indikator bahaya di pin D1 seketika menyala terang!
   - Lepaskan jari Anda dan biarkan suhu mendingin kembali ke bawah 31.0°C. LED indikator akan padam kembali secara otomatis.









7.6 Latihan

Latihan 1: Analisis Jaringan Internal Docker & Service Discovery

1. Mengapa penulisan localhost pada Node-RED di dalam kontainer Docker menyebabkan kegagalan koneksi (*Connection Refused*)?
    - Di dalam ekosistem Docker, setiap kontainer memiliki *network namespace* (ruang nama jaringan) yang terisolasi.
    - Jika Node-RED dikonfigurasi menggunakan alamat localhost atau 127.0.0.1, Node-RED akan mencari layanan MQTT pada kontainernya sendiri, bukan pada kontainer mqtt-broker. Karena di dalam kontainer Node-RED tidak ada layanan Mosquitto yang berjalan pada port 1883, koneksi akan ditolak dan mengakibatkan error *Connection Refused*.
2. Bagaimana mekanisme Docker Embedded DNS Server dan Bridge Network bekerja?
    - Ketika kontainer-kontainer berada di dalam jaringan *Docker Bridge* yang sama, Docker secara otomatis menjalankan Embedded DNS Server internal.
    - Saat kontainer Node-RED mencoba mengakses nama host seperti mqtt-broker (atau mqtt), Docker DNS server akan menerjemahkan (*resolve*) nama teks

tersebut menjadi alamat IP internal kontainer mqtt-broker yang valid di dalam *bridge network*. Dengan demikian, komunikasi antarkontainer dapat terjalin secara transparan tanpa harus mengetahui alamat IP fisik atau lokal laptop.

Latihan 2: Penanganan Masalah Konektivitas Jaringan & Subnet Firewall

1. Mengapa NodeMCU tidak berhasil memublikasikan data ke broker Mosquitto?
    - Kedua perangkat berada di subnet atau jaringan lokal (LAN) yang berbeda (NodeMCU berada di subnet 192.168.43.x dari *hotspot* seluler, sedangkan laptop berada di subnet 192.168.1.x dari jaringan rumah/Wi-Fi lain).
    - Tanpa adanya aturan *routing* khusus, gerbang (*gateway*) antar-subnet akan memblokir komunikasi langsung, sehingga NodeMCU tidak dapat menjangkau alamat IP broker laptop (192.168.1.15).
2. Langkah teknis agar kedua perangkat berada dalam domain broadcast/subnet yang sama:
    - Menyamakan Titik Akses (Access Point): Hubungkan laptop dan NodeMCU ke jaringan Wi-Fi atau *hotspot* yang persis sama (misalnya, hubungkan laptop ke *hotspot* HP yang sama dengan NodeMCU, atau sebaliknya).
    - Penggunaan Public MQTT Broker (Alternatif Cloud): Jika jaringan lokal tidak memungkinkan untuk disatukan, gunakan broker MQTT berbasis *cloud* (seperti HiveMQ public broker atau EMQX Cloud) sehingga kedua perangkat dapat berkomunikasi melalui internet publik menggunakan port MQTT.

Latihan 3: Implementasi Filter Histeresis pada Rule Engine

Untuk mencegah terjadinya *flickering* (kedipan cepat) akibat fluktuasi suhu di sekitar satu titik batas, kita menggunakan logika histeresis dengan dua ambang batas (atas dan bawah).

Berikut adalah kode JavaScript yang dapat Anda masukkan ke dalam Function Node:

JavaScript

```
// Memastikan variabel status LED tersimpan secara global antar-eksekusi (context)
let ledState = context.get('ledState') || 0;
let suhu = msg.payload.suhu;

// Logika Histeresis
if (suhu > 31.5) {
    ledState = 1; // Nyalakan LED jika melewati batas atas
} else if (suhu < 30.0) {
    ledState = 0; // Padamkan LED jika turun di bawah batas bawah
}
```

// Jika suhu berada di antara 30.0 dan 31.5, status ledState dipertahankan (mencegah flickering)

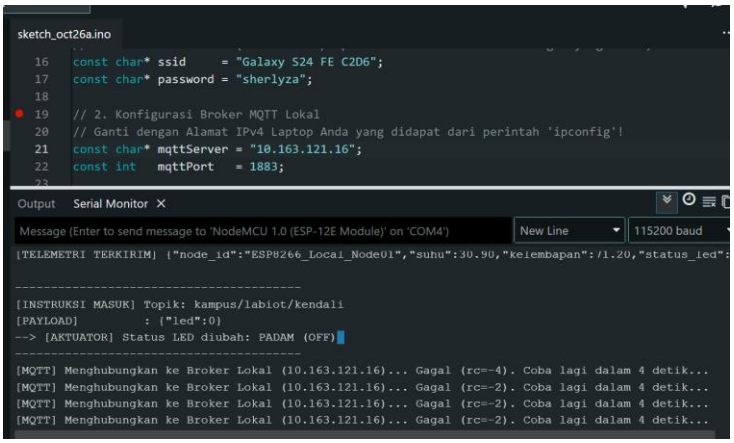
context.set('ledState', ledState);

msg.payload = { "led": ledState };

return msg;

Latihan 4: Analisis Mekanisme Volume Mount & Persistensi Data

1. Apa yang terjadi jika kontainer dimatikan dengan docker compose down, lalu dinyalakan kembali tanpa baris volume?
    - Seluruh data yang tersimpan di dalam direktori internal kontainer (seperti alur kanvas Node-RED yang telah Anda buat dan data basis data Mosquitto) akan hilang secara permanen.
    - Saat kontainer di-*restart* atau dibuat ulang (*re-created*), Node-RED akan kembali ke kondisi awal yang kosong (*clean state*) karena penyimpanan internal kontainer bersifat sementara (*ephemeral*).
2. Perbedaan mendasar antara Bind Mount dan Named Volume pada Docker:
    - Bind Mount: Menghubungkan direktori atau berkas tertentu dari sistem operasi host (laptop Anda) ke dalam kontainer (contoh: ./nodered:/data). Jalur direktori ditentukan secara eksplisit oleh pengguna di komputer lokal.
    - Named Volume: Dikelola sepenuhnya oleh Docker di dalam area penyimpanan khusus Docker di dalam sistem host (biasanya di /var/lib/docker/volumes/). Volume diberi nama khusus, lebih aman dari manipulasi sistem file luar, dan sangat direkomendasikan untuk pengelolaan data produksi yang bersih.



7.7 Tugas: Pengamanan Server IoT Industri (MQTT Authentication & Access Control)

Instruksi Pengerjaan Langkah demi Langkah:

1. Membuat Berkas Kredensial Terenkripsi di Dalam Kontainer:

Buka terminal/PowerShell di laptop Anda, lalu jalankan perintah berikut untuk mengeksekusi utilitas mosquitto_passwd langsung di dalam kontainer Mosquitto yang sedang berjalan:

docker exec -it mqtt-broker mosquitto_passwd -c /mosquitto/config/password.txt admin

- Sistem akan meminta Anda memasukkan kata sandi. Masukkan kata sandi: **rahasia123** (teks tidak akan muncul di layar saat diketik), lalu tekan Enter dan konfirmasi ulang kata sandi tersebut.
- Buka folder iot-server/mosquitto/config/ pada laptop Anda, pastikan kini terdapat berkas baru bernama password.txt yang berisi baris *hash* kredensial akun admin.

2. Memperbarui Konfigurasi Kebijakan Mosquitto:

Buka berkas iot-server/mosquitto/config/mosquitto.conf di editor teks Anda:

- Ubah nilai allow_anonymous true menjadi:

allow_anonymous false

- Tambahkan baris pemanggilan berkas kata sandi:

password_file /mosquitto/config/password.txt

- Simpan berkas mosquitto.conf.

3. Memuat Ulang Layanan Broker:

Restart kontainer mqtt-broker melalui terminal agar kebijakan keamanan baru diterapkan:

docker restart mqtt-broker

4. Analisis Dampak Kegagalan Sistem (Observasi Masalah):

- Buka antarmuka Node-RED di browser: Amati bahwa di bawah node mqtt in dan mqtt out, indikator status kini berubah menjadi warna merah bertuliskan **disconnected** atau **connection refused**.
- Buka Serial Monitor NodeMCU: Amati bahwa koneksi mikrokontroler kini gagal dan berulang kali menampilkan kode penolakan:
- [MQTT] Menghubungkan ke Broker Lokal... Gagal (rc=5). Coba lagi dalam 4 detik...

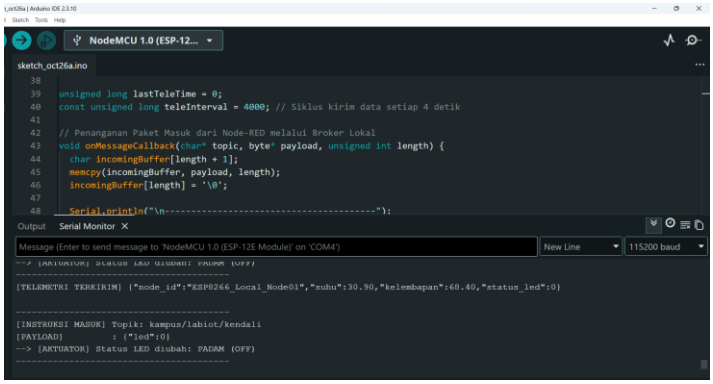
(Catatan: Kode status rc=5 merupakan representasi standar MQTT untuk 'Connection Refused: Not Authorized').

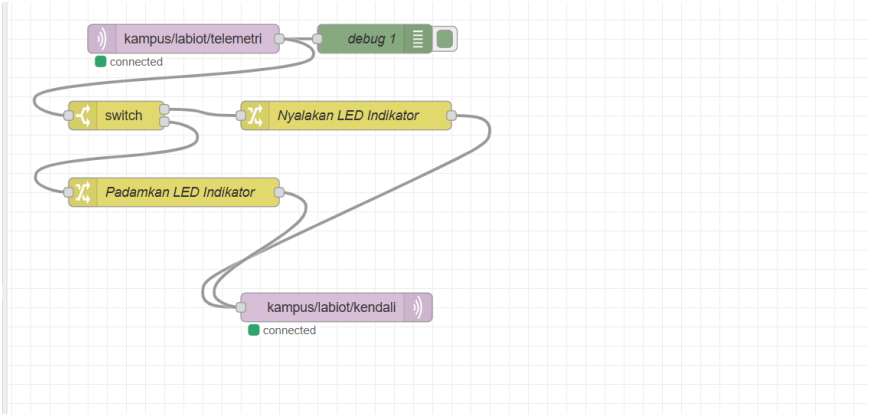
5. Rekayasa Perbaikan Sistem (Penyelesaian Tugas):

1. **Pemutakhiran Skrip Program NodeMCU ESP8266:**
    - Buka sketsa program Arduino IDE dari Praktikum 2.
    - Modifikasi bagian pemanggilan fungsi client.connect() di dalam fungsi reconnectMQTT() dengan menyertakan kredensial resmi:
    - // Format: client.connect(clientId, username, password)
    - if (client.connect(clientId, "admin", "rahasia123")) {
    - Serial.println(" Terhubung dengan Autentikasi Sah!");
    - client.subscribe(topicSubCmd);

}

    - Unggah ulang kode ke NodeMCU dan amati bahwa ESP8266 kini kembali berhasil terhubung (*rc=0*).
2. **Pemutakhiran Konfigurasi Kredensial pada Node-RED:**
    - Pada kanvas Node-RED, klik ganda node mqtt in.
    - Klik ikon pensil ( ) di samping nama broker Mosquitto Local.
    - Buka tab **Security** di bagian atas dialog konfigurasi.
    - Masukkan **Username:** admin dan **Password:** rahasia123.
    - Klik tombol **Update**, lalu klik **Done**, dan tekan tombol merah **Deploy**.
    - Amati bahwa indikator node MQTT pada kanvas Node-RED seketika kembali berwarna hijau (**connected**), dan aliran data telemetri serta kendali otomasi indikator LED loop tertutup kembali berfungsi normal dan aman dari gangguan pihak luar!




kampus/labiot/telemetri
connected
debug 1
switch
Nyalakan LED Indikator
Padamkan LED Indikator
kampus/labiot/kendali
connected